ChatGPT

Section 1: Claims

1. **AGREE (major).** The arXiv paper *"Self-evolving network verifiers"* (2608.11340, Aug 2026) indeed exists and uses Batfish and Containerlab-emulated FRR as oracles. It explicitly describes a loop where a learned symbolic verifier is tested against a real router oracle (Batfish or FRR)【8†L379-L387】. Thus it duplicates the differential-testing apparatus. We found no prior work quantifying *P(harm | verifier PASS)* or general verifier–emulator disagreement. (Search of related literature and blogs was inconclusive.) If this prior paper did exactly the same comparison, it undercuts novelty of the "first to compare verifier vs emulator" claim.

2. **AGREE (minor).** Each of (a)–(d) appears novel in networking context. (a) No published study has systematically reported empirical *P(harm | verifier PASS)* across multiple topologies. (b) No one has swept spec size vs false-pass rate. (c) Conformal risk control has been proposed in ML (ICLR 2024【52†L2-L10】) but not applied to network change management or verification. (d) An "autonomy–risk curve" is a new framing here. These ideas are not inherently trivial, though alone they might be seen as incremental. None is so weak as to be unpublishable if executed well, but the novelty hinges on framing rather than breakthrough theory.

3. **DISAGREE (major).** The spec-completeness sweep is clever but doesn't fully answer the critique. It will illustrate that adding properties lowers false-pass rate, but because the subsets are nested and use the same candidates repeatedly, the results are correlated. The sweep is essentially multiple hypothesis tests on the same data, so naive interpretation can mislead. One cannot treat each subset's rate as independent (and certainly not apply usual confidence intervals or p-values). At minimum this requires a multiple-comparison correction (e.g. Bonferroni or FDR) and acknowledgement of dependence. The approach shows a trend but does not *statistically* prove that "residual risk" is irreducible vs spec incompleteness. A reviewer would remain concerned that the high false-pass rate is just due to using a weak hand-crafted spec.

4. **AGREE (minor).** The sample-size arithmetic is correct: for $p \approx 0.1$, ±5% at 95% needs ~138 samples per bin, so 10 bins ≈1380 total, which is costly. Reducing to 4 bins (≈345 per bin) widens the interval to ±3.1%, which is acceptable. Using fewer bins or a continuous reliability curve is wise. One could also consider smoothing methods: e.g. plotting a calibrated probability curve with bootstrap confidence bands instead of fixed bins, or using a Bayesian credible interval on each bin's rate. Another alternative is to report a single "Expected Calibration Error" with confidence. Four bins is a reasonable tradeoff given budget, though one could consider 5–6 bins if flexibility allows.

5. **AGREE (minor).** It is true that isotonic regression often overfits on small samples (well under 1000) and Platt (sigmoid) is safer【44†L25-L34】【19†L249-L257】. In fact, *beta calibration* (Kull et al., AISTATS 2017) is an even better parametric method for n≈170【44†L25-L34】【19†L249-L257】. Temperature scaling is essentially a single-parameter Platt (used in multi-class softmax) and wouldn't offer much here. A fully Bayesian calibration (treating the probability as a beta posterior) could also improve uncertainty at small n. The plan's nested CV calibration is sound: it mirrors sklearn's CalibratedClassifierCV, which pools out-of-fold predictions to fit calibrators on held-out data【47†L194-L202】. Pooling those calibrated predictions is unbiased, so no fatal bias is introduced. This approach is legitimate and avoids wasting data on a fixed hold-out.

6. **AGREE (major).** Calibrating at the incident (cluster) level is a reasonable attempt to restore exchangeability by treating each fault scenario as one unit. In group-exchangeable conformal literature this is akin to "block" or "cluster" conformal prediction. For example, one could use a *Mondrian* or group-wise split conformal approach by conditioning on incident identity【23†L23-L32】. Specific methods exist (e.g. Romano et al. 2024 split-conformal for non-exchangeable data【28†L55-L64】) that basically say: use each incident's score once. The plan correctly notes that with ~20 incidents, α cannot be much below 1/(n+1)≈0.05. So indeed α≈0.05 is about the lowest one can target (e.g. α=0.05 requires allowing one exceedance in ~21 clusters). Thus the α claim is correct. The plan should explicitly cite a "group-exchangeable" conformal method (e.g. Angelopoulos & Bates 2022 or Oliveira et al. 2024) to justify treating incidents as units.

7. **AGREE (minor).** Rejecting a CNN on adjacency-matrix is correct: there's no fixed node ordering and a plain CNN would misinterpret permutations. With only ~700 examples, a deep neural network (CNN or large GNN) would massively overfit. The inductive bias of GNNs is more appropriate, but GNNs also typically require thousands of examples to train well, far more than 700. A gradient-boosted tree (GBDT) on engineered graph features (centrality, path changes, etc.) is a sensible baseline for such small n. However, I *disagree* with completely dismissing a GNN: a shallow GNN (1–2 layers, weight decay) might still pick up relational patterns if tuned carefully. If one could pretrain a GNN on synthetic but related network graphs or use semi-supervised techniques, it might work. But as a first pass, the plan's GBDT with hand features is reasonable given sample size.

8. **AGREE (minor).** The idea to trade "topologies × incidents" for more topologies and fewer incidents each is reasonable if emulator cost truly scales by candidate. One downside is that with only 5 incidents per topology, the per-topology risk estimate becomes noisier. If faults interact with topology in complex ways, fewer incident samples could underrepresent some failure modes. However, doubling the variety of topologies should improve generalizability and gives more folds for leave-one-topology-out validation. The trade-off is acceptable, but the write-up should note that incident-level confidence may be lower with fewer replicates.

9. **AGREE (minor).** 150 Batfish-FAIL candidates is a conservative choice. With 0 observed false-passes, the 95% Clopper–Pearson upper bound is about 1.97% (rule-of-three gives ≈2%). The rule-of-three is a classic heuristic for 0 events. A reviewer might nitpick that a binomial exact bound is more precise, but numerically they agree. If 2% is an acceptable upper bound, 150 is fine. If one needed <1%, more samples are needed. But the reasoning is sound: 0/150 yields ~0.02.

10. **DISAGREE (major).** Mapping *NO_CONVERGENCE* to harmful makes sense (it indicates a failure to achieve a stable state). However, excluding *UNDETERMINED* and *EXECUTION_FAILED* cases entirely is questionable. Dropping those outcomes throws away data. If any of those runs would have been harmful (e.g. emulator crash due to a pathological config), the harm rate is underestimated by excluding them. It biases *downward*: the measured P(harm | verifier PASS) is lower than the true rate. A safer approach is to treat these as harmful (or at least analyze them separately). The plan's code currently classifying them as safe=0 is definitely incorrect. So the treatment should be reconsidered.

11. **DISAGREE (major).** A purely negative result ("nothing predictably learnable; use trivial rule") is unlikely to be a full NOMS/CNSM/TNSM paper. These venues expect a substantive positive contribution. Merely showing *"we tried ML and it failed"* is typically considered a weak outcome. Unless the negative finding is exceptionally surprising or the bound is very novel, this reads like

a workshop or short paper at best. The pilot's trivial rule (oracle-only property fails ⇒ harmful) suggests little structure to exploit. Without a successful predictive model, the paper risks being an audit rather than a research contribution. Unless the authors can reframe the contribution (e.g. by proving impossibility or focusing on the conformal bounding method itself), it would likely be seen as insufficient for a major venue.

Section 2: A, B, C

A. Top three likely wrong aspects (ranked):
1. *Novelty claim*: It may already be negated by the arXiv 2026 "self-evolving network verifiers" paper. That prior art covers much of the same experiment apparatus (Batfish vs FRR)【8†L379-L387】, so the claim of novelty is doubtful (major).
2. *Learnability outcome*: The plan assumes a learnable residual risk model, but the pilot and Stage 1 show none. It's very plausible the final result is negative (no predictive signal beyond trivial rule), making the main "learn to predict risk" angle false (major).
3. *Specification issue/confounds*: Using only 5 of 12 oracle properties (hand-written) is a weak base. The false-pass measure might just show spec weakness, not something inherent to verification. The proposed "spec sweep" is not a rigorous fix (major).

B. Missing considerations:

- **Data validity**: All topologies and repairs are synthetic (one generator, parameterized ops). Real network configs or different generation methods might behave differently. The plan doesn't address how representative the dataset is.
- **Outcome variance**: Using only one emulator run per candidate means label noise. The plan mentions some replicates, but it should systematically consider label uncertainty (e.g. run each scenario multiple times to estimate stability, since FRR convergence can be nondeterministic).
- **Alternate harm measures**: Only "reachability harm" is considered. There may be more nuanced harms (e.g. performance degradation, transient loops) the spec can miss; the plan doesn't consider if "safe" means truly safe.
- **Operator realism**: Repairs are generated by scripted operators, not actual network fixes. If those operators have biases, the study might miss real-world repair patterns.
- **Benchmarking**: The plan lacks a baseline (besides trivial rule). Perhaps comparing to simpler heuristics or existing tools (e.g. checking differences in config) could highlight how much is truly unpredictable.

C. Strongest rejection reason (and defense):
Rejection paragraph: "While the study tackles an important question, the core contribution appears minimal. The experiment is essentially a detailed audit that reaffirms a trivial insight: if the verifier's spec omits a property, then updates can violate that property undetected. The novelty is undercut by recent work (e.g. the arXiv 'self-evolving verifiers' paper) that already performs Batfish vs router-emulator comparisons【8†L379-L387】. The learning models consistently fail (per the pilot), implying no new predictive signal exists beyond the obvious "oracle-only failures cause harm" rule. This yields a negative result with no positive takeaway, unlikely to meet main-track standards. In addition, the entire data methodology is fragile: using a single synthetic topology generator and scripted repairs, with only one emulator run per candidate, invites confounds and unreliability. Given these issues, the paper as planned would likely be rejected as lacking substantive new insight or generalizable conclusions."

The plan does *not* contain an explicit defense to such a critique. To survive, the authors would need to add substantial argument or data. For example, they must **explicitly compare to the "self-evolving"**

method, clarifying what is novel about their measurement. They should also address external validity: e.g. demonstrate that using diverse topology generators or real network configs does *not* change the outcome. They could preempt the "trivial rule" critique by showing that it fails on new cases or that bounding risk (via conformal) actually provides nontrivial value.

Stress-testing confounds: The plan should acknowledge that its synthetic nature may bias results. Using *one* topology generator means all graph structures have the same family; a different generator (or real network snapshots) might yield different P(harm|pass). Likewise, parameterized operators may not capture real operator behavior, so the distribution of updates may be unrealistic. Finally, a single emulator run per candidate ignores nondeterminism in FRR: a "harm" outcome could be a random fluke. Re-running each scenario multiple times (beyond the limited replicates already planned) would quantify this noise. Without addressing these, the experiment risks mistaking artifacts of the setup for general network behavior.

Section 3: Papers Table

Title	Venue / Year	URL	Why it matters	Verified?
Self-evolving network verifiers	arXiv 2026	https://arxiv.org/abs/2608.11340	Uses Batfish vs FRR containerlab oracles in a loop【8†L379-L382】, i.e. the same apparatus.	Y
Config2Spec: Mining Network Specifications from Network Configurations (Birkner *et al.*)	NSDI 2020	USENIX	Automatically extracts near-complete specs from configs (no false negatives)【48†L49-L57】; undermines "hand spec" novelty.	Y
Conformal Risk Control (Angelopoulos *et al.*)	ICLR 2024	ICLR Proc. PDF	Introduces conformal risk control for ML; relevant background on bounding expected loss【46†L12-L21】.	Y
Beta calibration: a well-founded improvement on logistic calibration (Kull *et al.*)	AISTATS 2017	MLR 54	Shows beta calibration often outperforms Platt on small data【44†L25-L34】 (relevant to claim on calibration).	Y

Section 4: Other Changes to the Plan

Aside from the points above, I would refine the plan in several ways. First, I would explicitly **validate the synthetic setup**: for example, run experiments on at least one real network topology (or a different synthetic model) to ensure results aren't an artifact. Second, I'd increase emulator replication: instead of a single run, each candidate could be tested 2–3 times to measure consistency of the *HARMFUL* label. Third, I'd consider additional features or tasks. For instance, besides reachability properties, track

convergence time or intermediate loops, to see if these indicate risk. Fourth, if relying on hand-engineered graph features, I'd derive or prune them systematically (to avoid cherry-picking); perhaps use feature-selection on a larger simulation dataset first. Finally, I'd explicitly plan for debiasing the spec-completeness analysis: use bootstrapping or holdout sets when evaluating different subsets of properties to avoid overfitting that comparison.